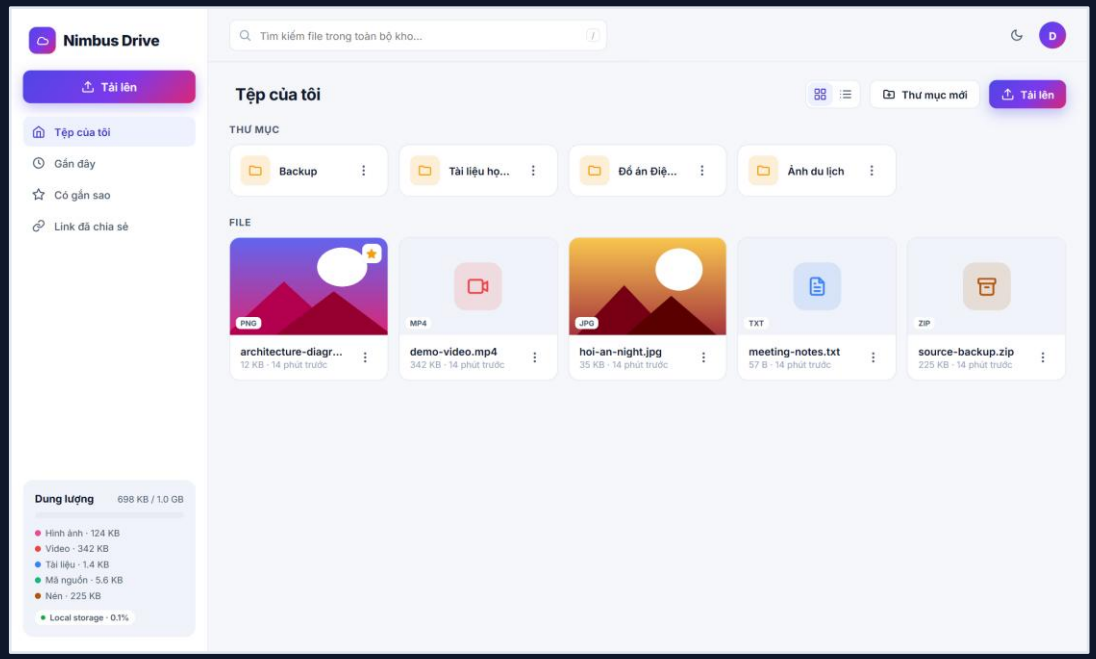
Nimbus Drive

Nền tảng lưu trữ file đám mây trên kiến trúc tương thích Amazon S3

Design and Deployment of a Spring Boot Containerized Cloud File Storage Platform based on AWS S3-Compatible Architecture with an Automated CI/CD Pipeline



Spring Boot 3.5 · PostgreSQL · S3 · Docker · Terraform · GitHub Actions

Sinh viên: [Họ tên] · MSSV: [MSSV]
GVHD: [Họ tên giảng viên] · Môn Điện toán đám mây

Nội dung trình bày

01

Bài toán & mục tiêu

Vì sao cần, làm gì

02

Kiến trúc hệ thống

S3-compatible, phân lớp, CSDL

03

Sản phẩm & demo

Tính năng, giao diện

04

Bảo mật

Các mối đe dọa và biện pháp

05

Container & CI/CD

Docker, Compose, GitHub Actions

06

Triển khai AWS & kết quả

Terraform, kiểm thử, hướng phát triển

Bài toán: lưu trữ dữ liệu linh hoạt, an toàn, tự động

Thực trạng

- Nhu cầu lưu trữ & chia sẻ file trực tuyến tăng mạnh
- Object storage chuẩn **S3** là nền tảng của mọi cloud drive
- Triển khai thủ công dễ lỗi, khó tái tạo môi trường
- Phụ thuộc một nhà cung cấp gây rủi ro (vendor lock-in)

1

Ứng dụng web hoàn chỉnh

Upload, thư mục, xem trước, chia sẻ, quota

2

Kiến trúc tương thích S3

Chạy trên AWS S3 hoặc S3 tự host, không đổi mã

3

Container hóa + IaC

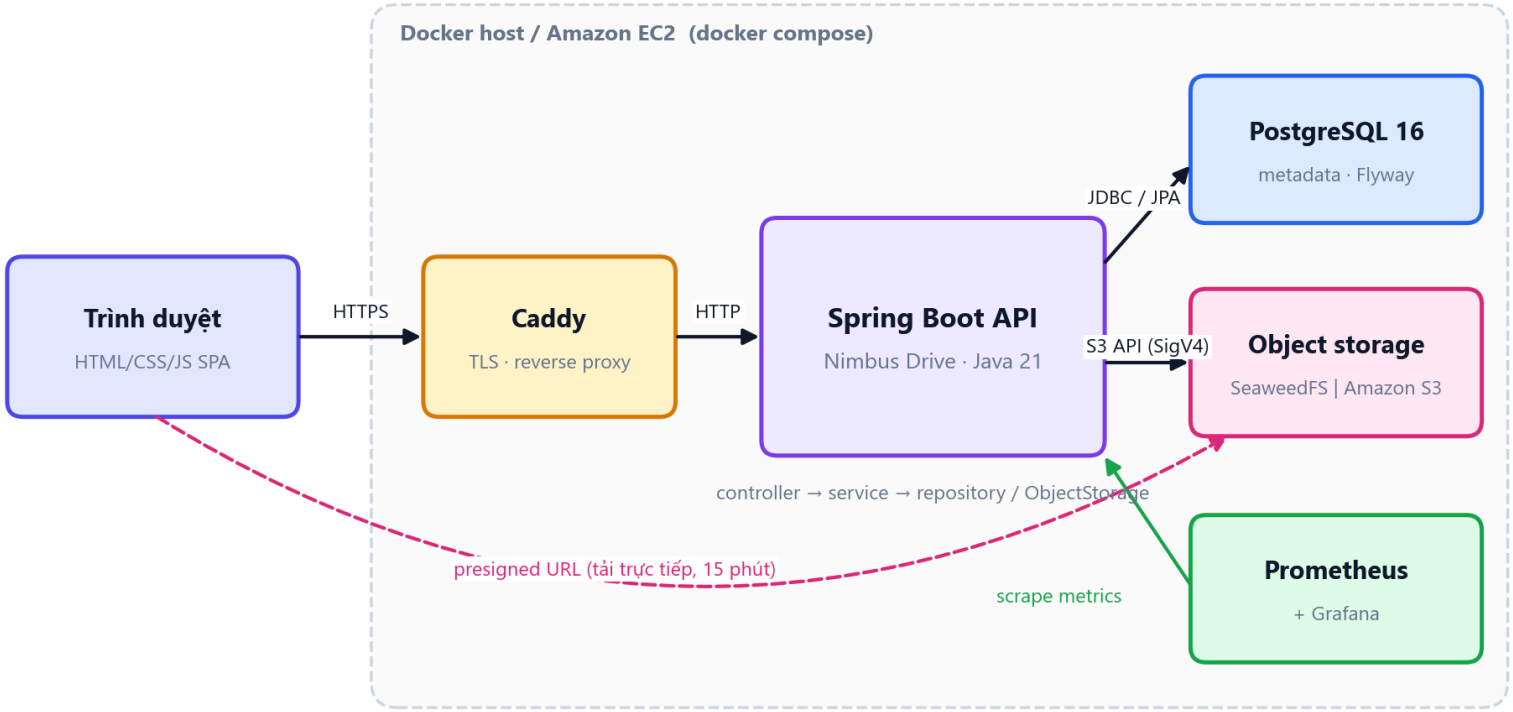
Docker Compose & Terraform cho AWS

4

CI/CD tự động

Test → image → scan → deploy → smoke test

Kiến trúc: tách metadata và nội dung file



PostgreSQL

Người dùng, thư mục ảo, metadata, link chia sẻ

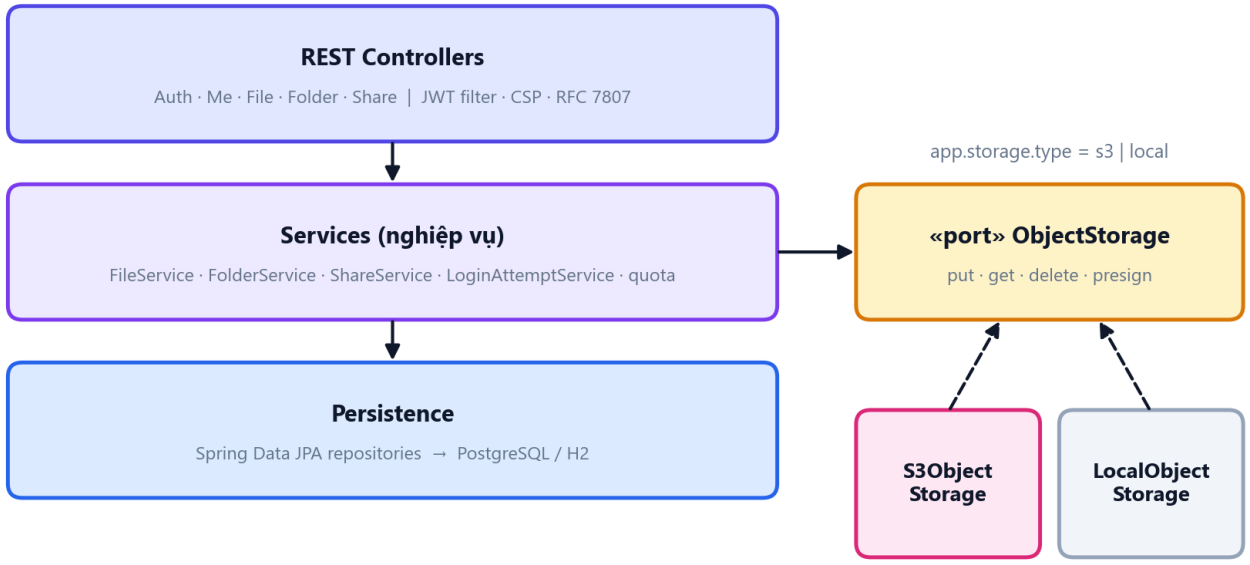
Object storage

Nội dung file, key phẳng owner/uuid

Presigned URL

Trình duyệt tải thẳng từ S3, giảm tải server

Ports & Adapters: một mã nguồn, nhiều nhà cung cấp S3

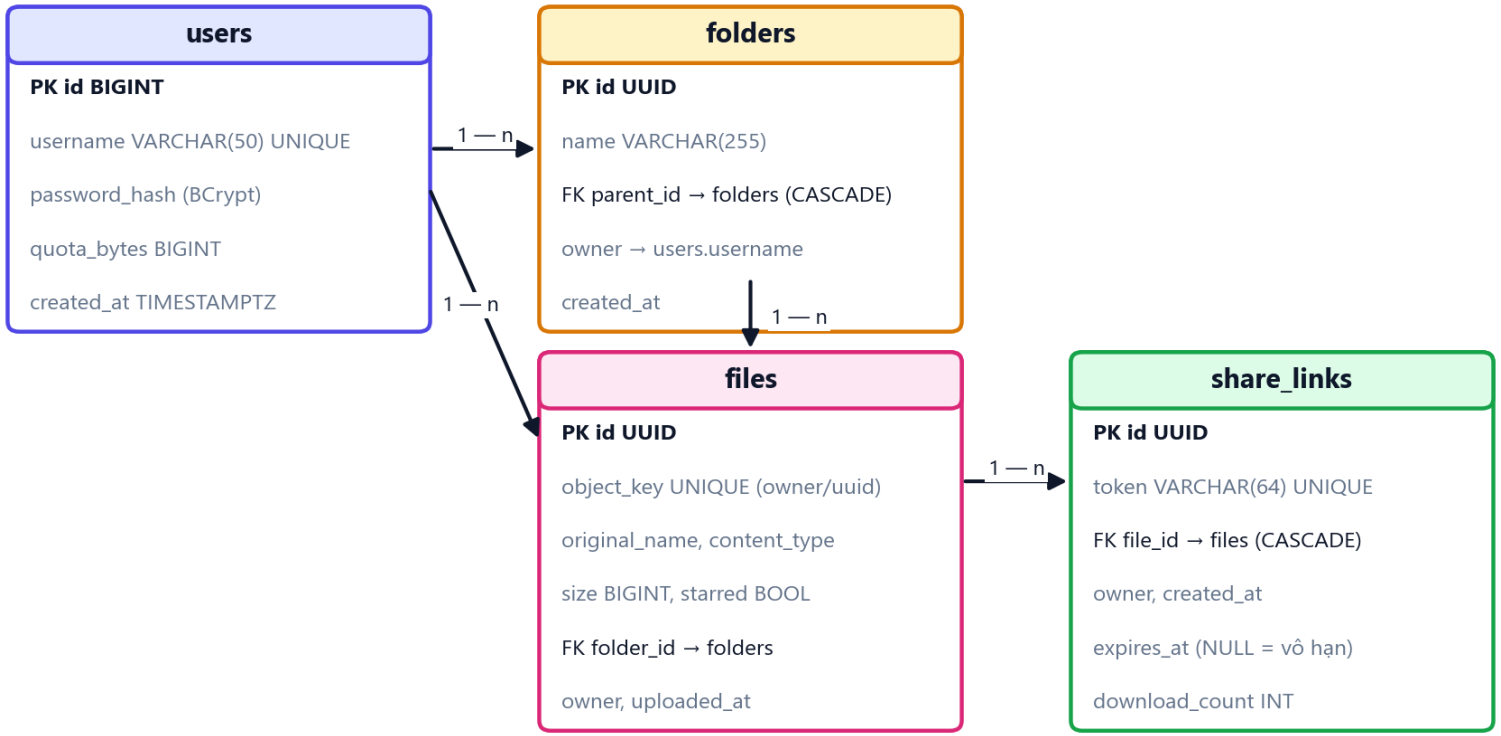


- Interface **ObjectStorage**: put · get · delete · presign
- **S3ObjectStorage** dùng AWS SDK v2 – chạy với AWS S3, SeaweedFS, Ceph, MinIO
- **LocalObjectStorage** cho dev/test không cần Docker
- Đổi backend chỉ bằng biến môi trường (12-factor)

Bài học thực tế

MinIO community ngừng phát hành bản dựng (410 Gone) → chuyển sang SeaweedFS mà không sửa dòng code nghiệp vụ nào.

Cơ sở dữ liệu: 4 bảng, quản lý bằng Flyway



- Thư mục dạng cây, parent_id **ON DELETE CASCADE**
- object_key duy nhất ánh xạ tới S3
- Token chia sẻ 192 bit, có hạn dùng
- Migration có phiên bản, chạy trên PostgreSQL & H2
- Index theo (owner, folder) cho truy vấn nhanh

SẢN PHẨM

Tính năng chính của Nimbus Drive

1

Tài khoản JWT

BCrypt, khóa tạm sau 5 lần sai

2

Upload kéo-thả

Nhiều file, thanh tiến độ, 100 MB/file

3

Thư mục lồng nhau

Breadcrumb, kéo file vào thư mục

4

Xem trước

Ảnh, PDF, video, audio, code

5

Chia sẻ link

Hết hạn 1h–30 ngày, đếm lượt, thu hồi

6

Presigned URL

Tải trực tiếp từ S3 15 phút

7

Quota & thống kê

1 GB/user, phân bố theo loại file

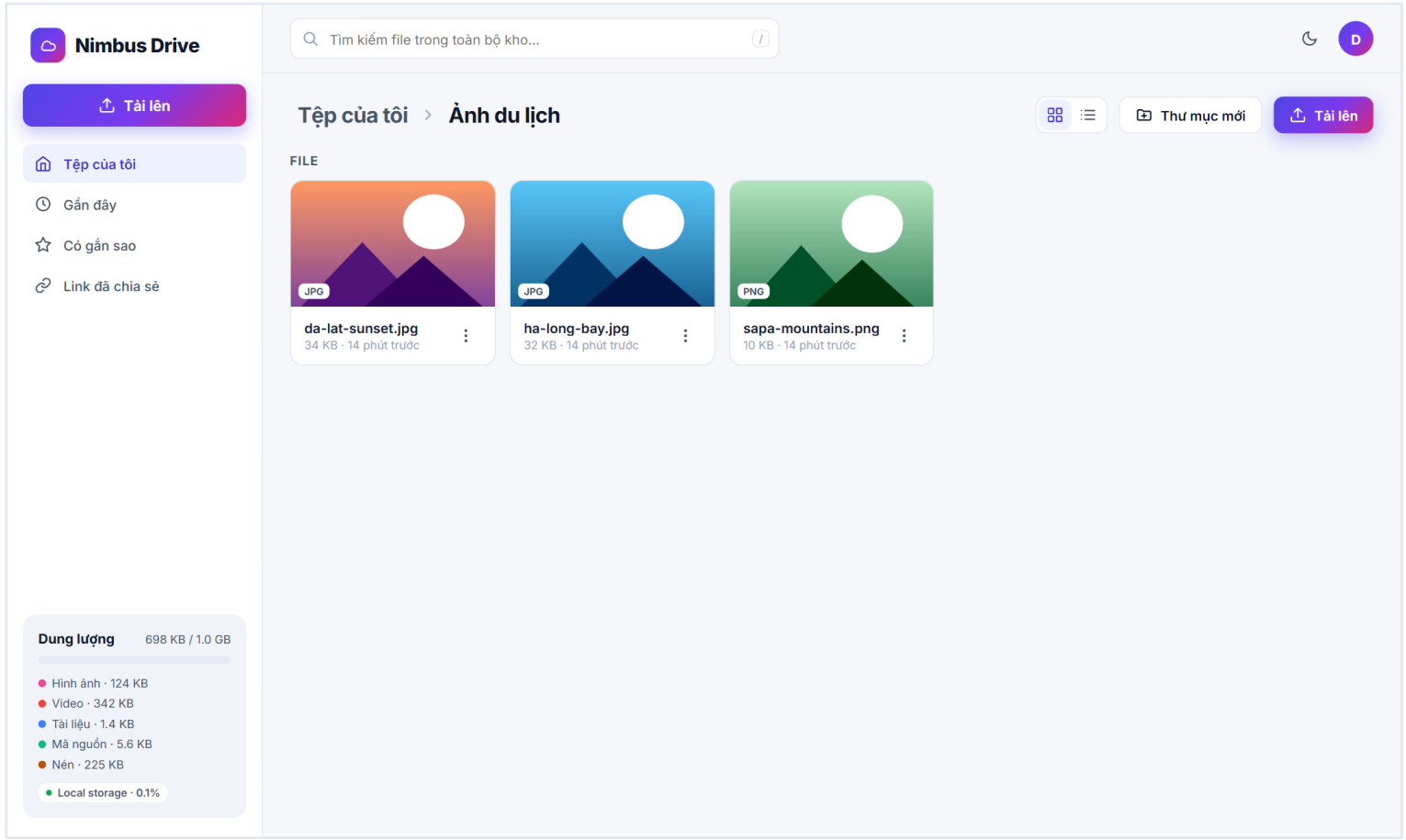
8

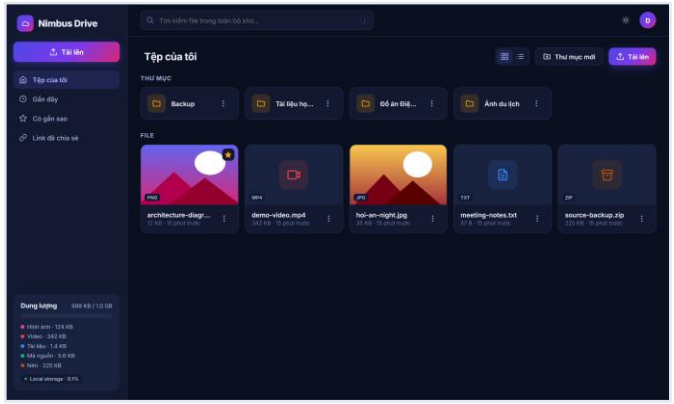
Trải nghiệm

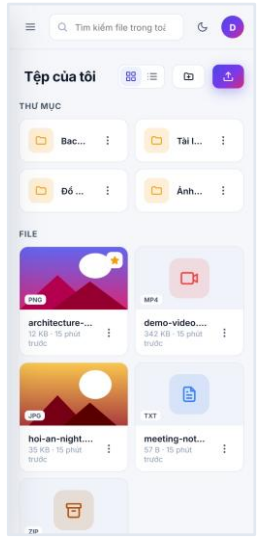
Tìm kiếm, gắn sao, dark mode, mobile

DEMO

Giao diện quản lý file



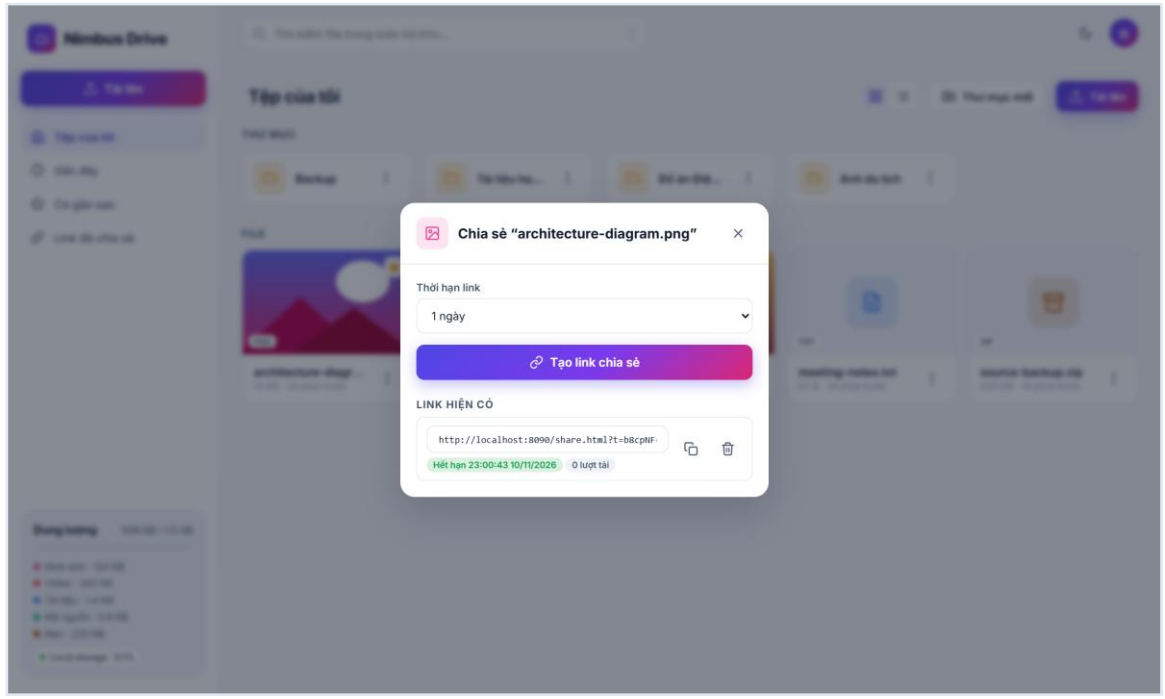




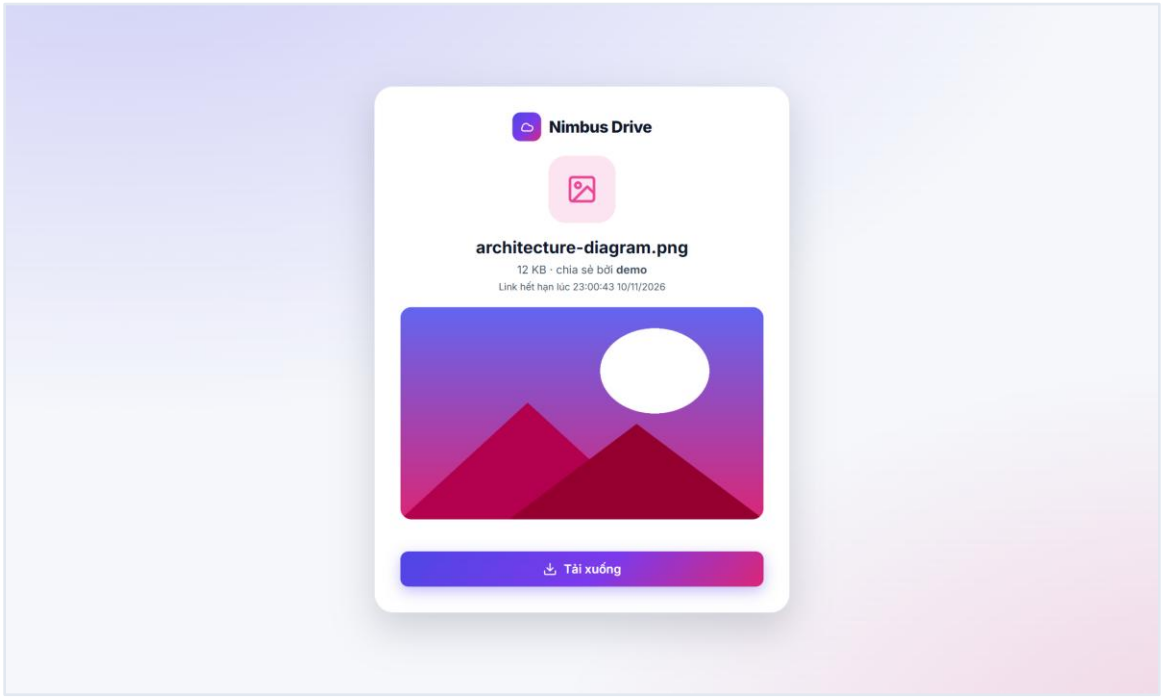
Lưới / danh sách · ảnh thu nhỏ · thống kê dung lượng · sáng/tối · responsive

DEMO

Xem trước và chia sẻ file



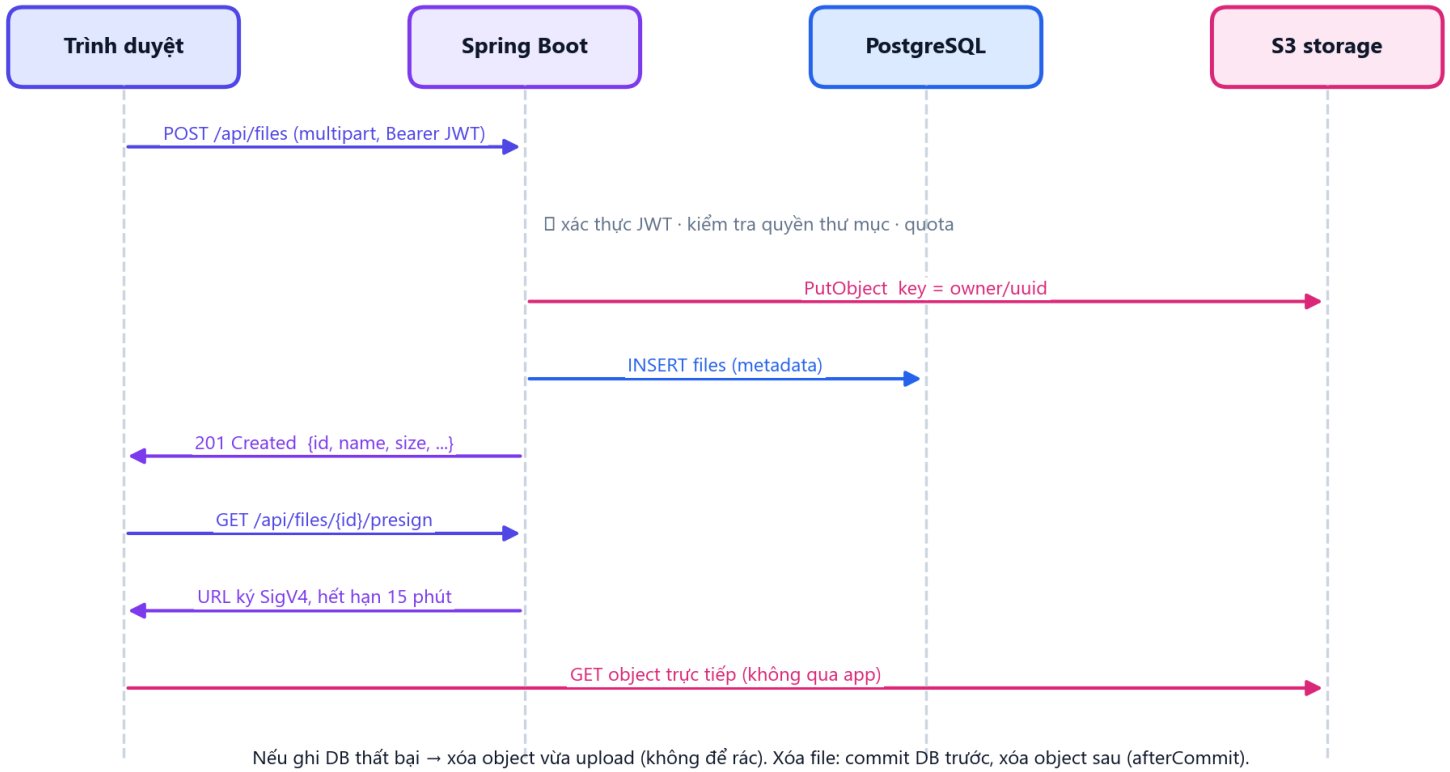
Tạo link có thời hạn, sao chép, thu hồi



Khách tải file không cần đăng nhập

File HTML/SVG do người dùng tải lên luôn bị ép tải về (không hiển thị inline) để chống XSS

Luồng upload và tải qua presigned URL



1. **Xác thực & kiểm tra quota**
2. **PutObject lên S3 trước**
3. **Ghi metadata; lỗi → xóa object**
4. **Xóa: commit DB rồi mới xóa object**

BẢO MẬT

Bảo mật nhiều lớp

Xác thực

JWT HS256 · BCrypt · khóa 5 phút sau 5 lần sai (429)

Phân quyền

Mọi truy vấn lọc theo owner; tài nguyên người khác → 404

Chống XSS

CSP script-src 'self'; file HTML/SVG không hiển thị inline

Đầu vào

Làm sạch tên file, chặn path traversal, validate DTO

Hạ tầng AWS

IAM role quyền tối thiểu, IMDSv2, không lưu access key

Dữ liệu

S3 chặn public, mã hóa SSE, versioning; HTTPS qua Caddy

Container hóa với Docker & Docker Compose

```
# Stage 1: build (JDK + Maven, cache .m2)
FROM maven:3.9-eclipse-temurin-21 AS build
RUN mvn package && java -Djarmode=tools \
    -jar app.jar extract --layers

# Stage 2: runtime (chỉ JRE)
FROM eclipse-temurin:21-jre
COPY dependencies/ application/ ./
USER appuser        # không chạy root
HEALTHCHECK curl /actuator/health/liveness
```

app

Spring Boot · chờ DB & S3 healthy

postgres

PostgreSQL 16 · volume

seaweedfs

S3 API :8333 tự host

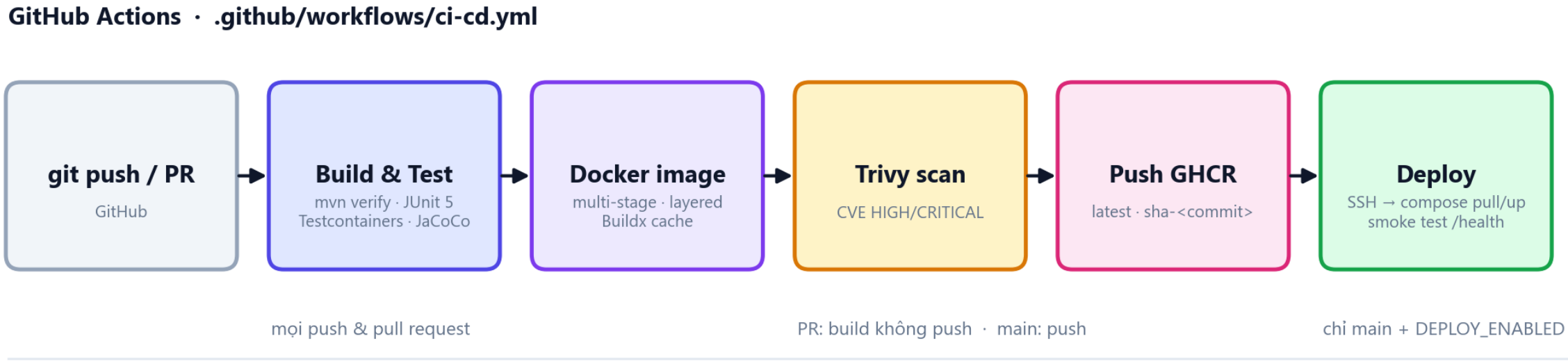
caddy

HTTPS tự động (profile proxy)

prometheus + grafana

Giám sát (profile monitoring)

Pipeline CI/CD tự động với GitHub Actions



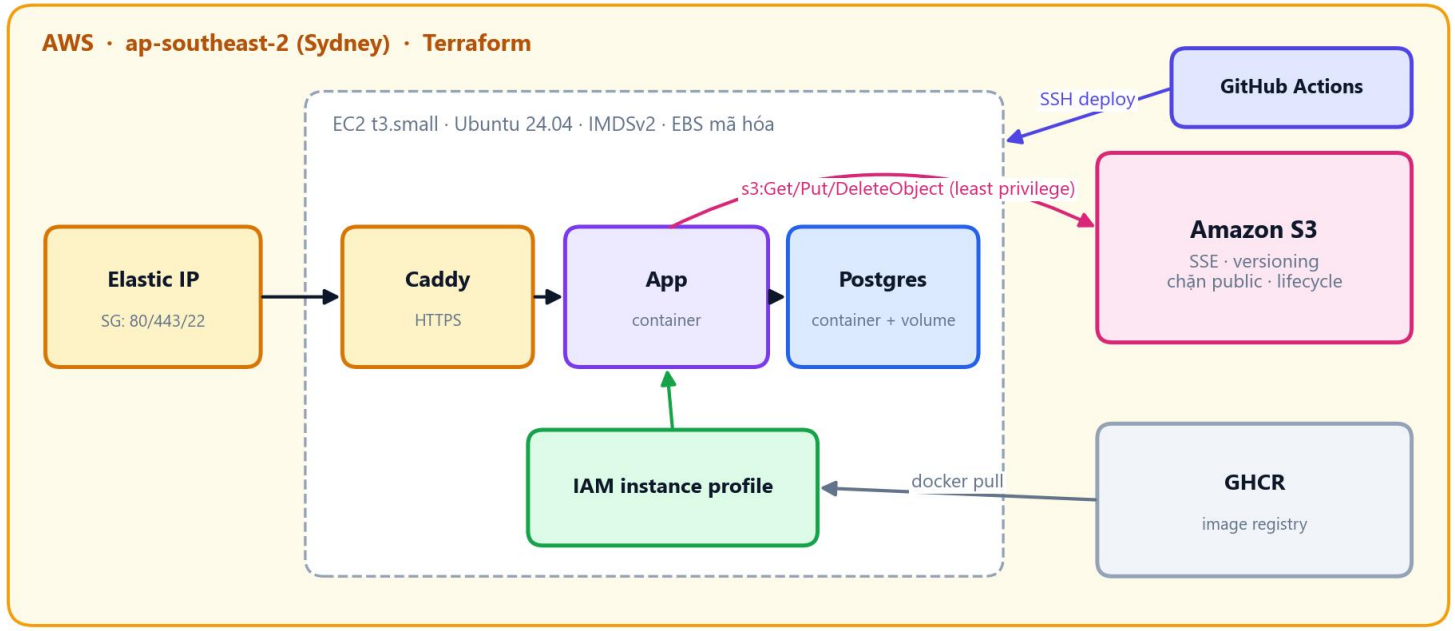
Fail ở bất kỳ bước nào → dừng pipeline; production luôn chạy image đã qua test, gắn tag theo commit (rollback = deploy lại sha cũ).

3 job: test → image → deploy

2 tag: latest + sha-<commit>

90s smoke test sau deploy

Triển khai trên AWS bằng Terraform



- **S3**: chặn public, SSE-AES256, versioning, lifecycle, CORS
- **IAM role** chỉ Get/Put/Delete trên 1 bucket
- **EC2** Ubuntu 24.04, IMDSv2, EBS mã hóa, Elastic IP
- user_data cài Docker, sinh secret ngẫu nhiên
- Đã chạy thật ở Sydney: upload, S3, presigned URL, chia sẻ đều đạt

KẾT QUẢ

Kết quả kiểm thử

22/22	~80%	22	5
ca test đạt trên CI	độ phủ dòng (JaCoCo)	REST endpoint	service container

Unit test	Tên file, phân loại, token, khóa đăng nhập, JWT
API test (MockMvc + H2)	Luồng đầy đủ: cô lập user, quota, thư mục đệ quy, chia sẻ, chống XSS
Integration (Testcontainers)	PostgreSQL 16 và SeaweedFS S3 thật – đạt trên GitHub Actions

KẾT LUẬN

Hạn chế và hướng phát triển

Hạn chế

- Upload qua server, giới hạn 100 MB/file
- Rate-limit lưu RAM, chưa dùng chung nhiều instance
- PostgreSQL chung EC2, chưa backup tự động
- Còn 2 lỗ hổng cần Spring Boot 4; AWS chưa có HTTPS

Upload trực tiếp S3

Presigned PUT + multipart, file nhiều GB

Thùng rác & phiên bản

Xóa mềm, khôi phục qua S3 versioning

Managed services

RDS Multi-AZ, ECS Fargate, CloudFront

Bảo mật nâng cao

OAuth2 Google, 2FA, quét virus, Redis

Cảm ơn thầy cô và các bạn đã lắng nghe

Hỏi & đáp · Demo trực tiếp

`docker compose up -d --build`	`http://localhost:8080`	`/swagger-ui.html`	`./mvnw verify`
Chạy toàn bộ hệ thống	Giao diện web	Tài liệu API	Chạy kiểm thử